Restaurant Rush

Assignment group	Group 1
Repository	https://github.com/JeremyChooCK/CT2003-Restaurant-Rush

Group members

Name	Matriculation no.
CHIANG JUN SIANG	[ ]
CHONG YAN QING	U2620471K
CHOO CHONG KAI JEREMY	[ ]
CHRISTOPHER TAN CHIN POH	U2620903B

Contents

1. Overview and how to run
2. Class diagrams (Stage 1, Stage 2, Stage 3 final)
3. Design discussion: abstractions, OO concepts, SOLID, coupling and cohesion
4. Trade-offs and an alternative design
5. How the player's choices change the result
6. Evolution across the three stages (GitHub evidence)
7. Functional evidence: rule checks and reproducible runs
8. Additional features
9. Reflection

1. Overview and how to run

Restaurant Rush is a turn-based game in which the player manages a small restaurant for 18 turns. Each turn the player decides what the Host (Stage 3), Waiter and Chef do. The Cashier works automatically. Customers choose their own food and lose patience while they wait. After turn 18 the player wins only if **at least 4 customers paid**, **revenue is at least $45.00** and the **average satisfaction of paid customers is at least 60**.

The game needs only JDK 17 or newer and works fully offline. Build with `build.bat` or `./build.sh`, play with `run.bat` or `./run.sh`, and run the 40 automated rule checks with `test.bat` or `./test.sh`. Every recorded run in this report can be replayed exactly, for example `java -cp out restaurantrush.Main --echo < runs/stage3-victory.txt`.

The code follows the **Boundary-Control-Entity** structure. `boundary` only reads input and prints the state. `control` holds the turn loop and all game rules. `entity` holds the restaurant's objects and keeps them consistent. No rule lives in the user interface.

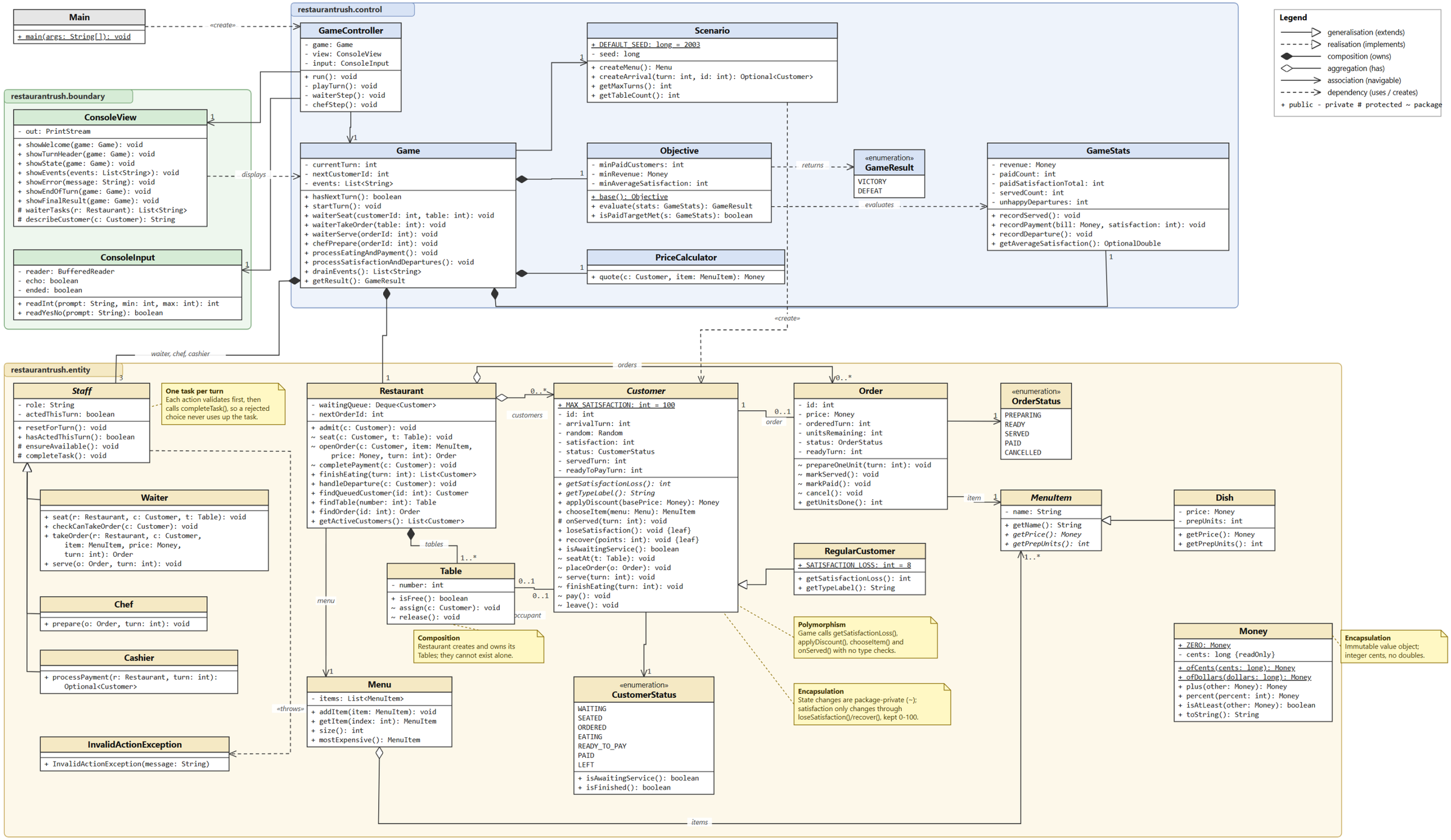
2. Class diagrams

The three diagrams follow on A3 landscape pages, one per stage. Classes new in a stage are outlined in red. Italic names and operations are abstract, underlined members are static, `{leaf}` marks final methods, and ~ marks package-private operations. Full-size PNG and SVG copies are in `docs/uml/` of the repository and at each tag. The Stage 3 diagram matches the submitted final code.

2.1 Stage 1 class diagram: basic restaurant operation

Restaurant Rush - Stage 1 class diagram (basic restaurant operation)

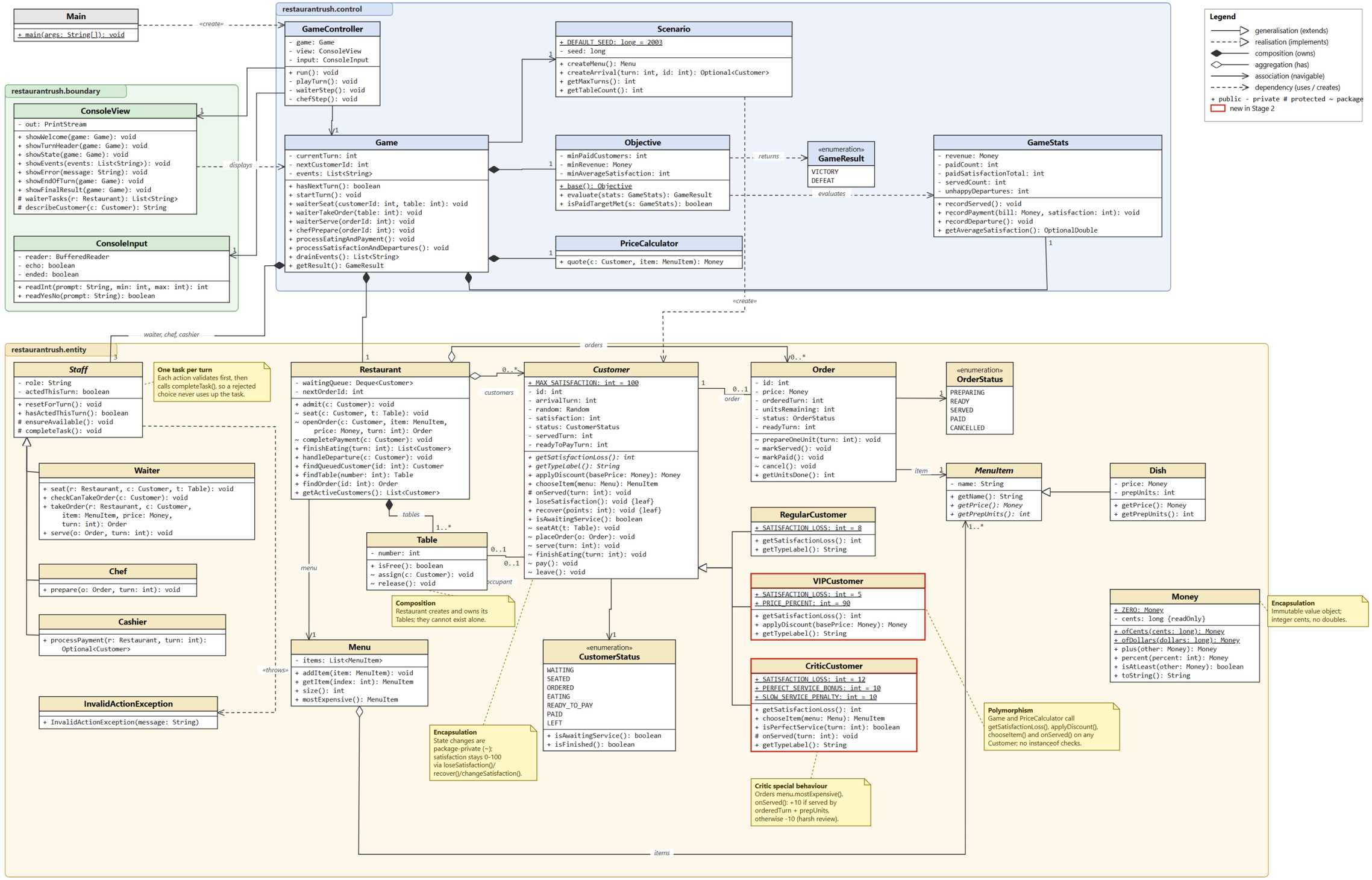
Boundary-Control-Entity packages. Italic = abstract, underlined = static, {leaf} = final. Money is the type of every price.



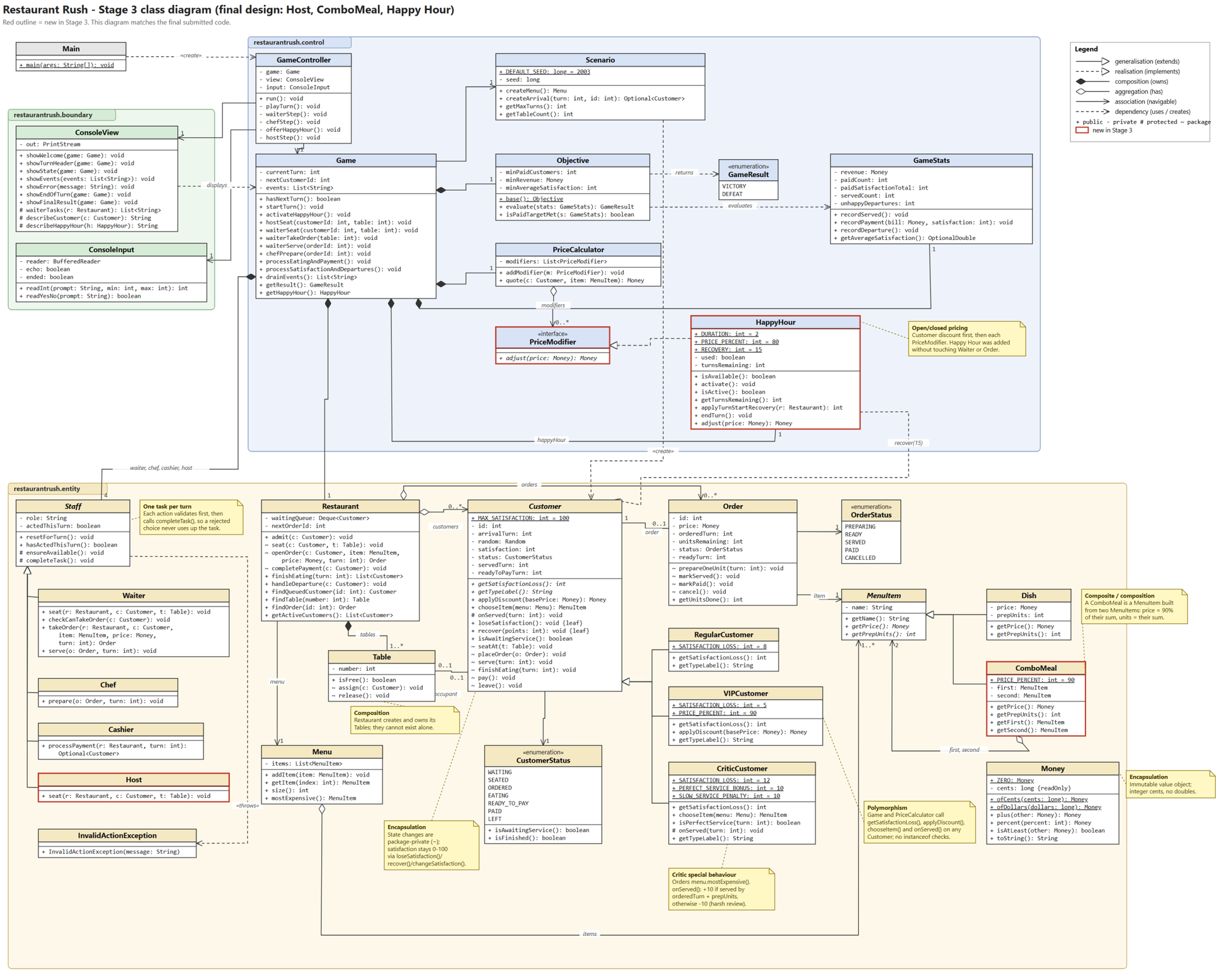
2.2 Stage 2 class diagram: customer behaviour (new classes outlined in red)

Restaurant Rush - Stage 2 class diagram (customer behaviour extension)

Red outline = new in Stage 2. Only Scenario (setup) changed among Stage 1 classes; Game only gained a clearer serve message.



2.3 Stage 3 class diagram: final design, matches the submitted code (new classes outlined in red)



3. Design discussion

3.1 Key abstractions and their responsibilities

Class	Kind	Responsibility
`Game`	control	Applies the turn rules: arrivals, staff actions, eating and payment, satisfaction loss and departures, Happy Hour timing. Turns the player's numeric choices into objects and reports what happened as event messages. No input or output.
`GameController`	control	Runs the 18-turn loop in the brief's order. Asks the view and input for each decision and retries when `Game` rejects a choice.
`Scenario`	control	All setup in one place: turns, tables, menu, arrival schedule and seed. Later stages register new customers and menu items here.
`PriceCalculator`, `PriceModifier`, `HappyHour`	control	Locks an order's price: customer discount first, then each restaurant-wide modifier. `HappyHour` is one modifier and also manages its two-turn duration and the +15 recovery.
`GameStats`, `Objective`	control	Running totals (revenue only from paid bills) and the victory check.
`Restaurant`	entity	Owns the tables, the waiting queue, customers and orders, and keeps them consistent when customers are seated, pay or leave.
`Customer` + subclasses	entity	The visit state machine and satisfaction. Subclasses supply patience, discount, food choice and reaction to service.
`Order`	entity	Locked price, preparation progress and status.
`MenuItem`, `Dish`, `ComboMeal`, `Menu`	entity	What can be ordered, its price and its preparation units.
`Staff` + `Waiter`, `Chef`, `Cashier`, `Host`	entity	Each enforces "at most one task per turn" and validates its own actions.
`Money`	entity	Immutable amount in integer cents with half-up rounding.
`ConsoleView`, `ConsoleInput`	boundary	Display the state, and read and format-check numbers and yes/no answers.

3.2 Object-oriented concepts in the code

Encapsulation. `Customer`, `Order` and `Table` expose read-only getters. Their state changes (`seatAt`, `placeOrder`, `serve`, `pay`, `leave`, `prepareOneUnit`) are package-private, so only the restaurant and its staff can move a customer through the visit, and the UI cannot. Satisfaction can only change through `loseSatisfaction()`, `recover()` and the protected `changeSatisfaction()`, which keep it between 0 and 100. `Money` is immutable and hides its cents, so a locked price cannot be altered later. `Staff` hides the "already acted" flag behind `ensureAvailable()` and `completeTask()`.

```
// Staff subclasses validate first and only then use up the task (Waiter.seat)
public void seat(Restaurant restaurant, Customer customer, Table table) throws InvalidActionException {
    ensureAvailable();                // throws if the Waiter already acted this turn
    restaurant.seat(customer, table);  // throws if the customer is not queued or the table is taken
    completeTask();                   // reached only when the action succeeded
}
```

Inheritance. `RegularCustomer`, `VIPCustomer` and `CriticCustomer` extend `Customer`. `Dish` and `ComboMeal` extend `MenuItem`. `Waiter`, `Chef`, `Cashier` and `Host` extend `Staff`, sharing the one-task-per-turn mechanism.

Polymorphism. The rules never check a customer's type. `Game` calls `customer.loseSatisfaction()`, which uses the subclass's `getSatisfactionLoss()`. `PriceCalculator` calls `applyDiscount()`. `Waiter.serve()` triggers `onServed()`, which only the Critic overrides. `Game` asks for `chooseItem(menu)`, which the Critic overrides to pick the most expensive item. Similarly, the Chef and `Order` only call `getPrepUnits()` and `getPrice()` on a `MenuItem`, whether it is a dish or a combo.

```
// Game.processSatisfactionAndDepartures(): one loop for every customer type
for (Customer customer : restaurant.getActiveCustomers()) {
    if (!customer.isAwaitingService()) { continue; }
    customer.loseSatisfaction();              // 8, 5 or 12 depending on the object
    if (customer.getSatisfaction() == 0) { restaurant.handleDeparture(customer); ... }
}
```

Composition and aggregation. `Restaurant` creates and owns its `Tables` (composition). `Game` owns its staff, statistics, objective, price calculator and Happy Hour. `ComboMeal` is built from two existing `MenuItems` (aggregation: the dishes also exist on their own). It is itself a `MenuItem`, a small Composite pattern, which is why combos needed no change to `Order` or `Chef`.

3.3 SOLID principles

Principle	Where it appears
Single responsibility	`ConsoleInput` only parses input, `ConsoleView` only formats output, `GameController` only sequences the turn and retries, `Game` only applies rules, `GameStats`/`Objective` only score, and `PriceCalculator` only prices. For example, changing the victory targets touches `Objective` alone.
Open/closed	Stage 2 added two customer types without modifying `Customer`, `Game`'s rules, `Waiter` or `Cashier`; only the `Scenario` setup changed. Stage 3 added combos without modifying `Order`, `Chef` or `Menu`, and Happy Hour pricing plugs into `PriceCalculator` as a `PriceModifier`. A future promotion would be one new class.
Liskov substitution	Any `Customer` subclass works wherever a `Customer` is expected. They keep the contract: loss is positive, a discount never raises a price, and satisfaction stays 0-100 because only the base class changes it. A `ComboMeal` can be ordered, cooked and priced exactly like a `Dish`. `Host` keeps `Staff`'s one-task rule.
Interface segregation	Only lightly reflected. `PriceModifier` is a single-method interface, so `HappyHour` implements only what pricing needs. We did not add interfaces where there was only one implementation.
Dependency inversion	`PriceCalculator` depends on the `PriceModifier` abstraction, not on `HappyHour`. Rules depend on the abstract `Customer`, `MenuItem` and `Staff`. (`GameController` still uses the concrete console classes; see trade-offs.)

3.4 Coupling and cohesion

Dependencies point one way: boundary → control → entity. The entity package does not import anything from control or boundary, so it compiles on its own. Classes are cohesive: each entity looks after its own state, and each control class has one job. The main coupling hotspot is `Game`, which coordinates many entities. We accepted this because it is the single place that knows the turn order. It acts as a façade, so the controller and view only talk to `Game` and read entity getters.

4. Trade-offs and an alternative design

Decision	Benefit	Cost
Package-private state changes in the entity package	Strong encapsulation: only staff and the restaurant can change a customer's state.	All entity classes must live in one package.
Extension hooks in `Customer` from Stage 1 (`applyDiscount`, `chooseItem`, `onServed`)	Stage 2 needed no change to `Customer` or the rules.	Stage 1 contains two hooks that Regular customers do not override.
`Game` returns event messages as strings	Simple, and the view just prints them.	`Game` holds English text; a GUI would prefer event objects.
Happy Hour wired into `Game` and `GameController` directly	Simple and easy to follow.	Stage 3 had to modify those two classes. A generic "turn listener" mechanism would avoid that but was more than this game needs.
Each customer has its own seeded random sequence	Choices are reproducible, even if the player makes and corrects mistakes.	Food choice is not influenced by game state.
Concrete console classes in `GameController`	Less code and no unnecessary interfaces.	A GUI would need a small view interface to be extracted.

Alternative considered: a `CustomerType` enum with `switch` statements. A single `Customer` class with a type field, where `Game` and `PriceCalculator` switch on the type to decide the loss, discount and special behaviour. It is shorter for three types, but every new type would require editing the turn rules in several places and testing them again. That breaks the open/closed principle and the brief's requirement that behaviour be dispatched through the customer objects. We chose subclasses, so Stage 2 only added two files.

We also considered the Command pattern for staff actions (one class per action). It would allow undo and logging, but the game has only a few actions and needs no undo, so plain methods on `Game` were clearer.

5. How the player's choices change the result

Every customer gets the same 18 turns, but the staff are the bottleneck. With one Waiter and an arrival every three turns, seating, taking the order and serving already fill the Waiter's turns. Any delay costs satisfaction, and a late customer may not pay before turn 18.

- **Order of tasks.** In Stage 1, seating each customer on arrival, taking the order next turn while the Chef cooks it, and serving the turn after gives five paid customers at 84 satisfaction each (victory, $57.00). A manager who stops after the first customer ends with one payment and a walk-out (defeat).
- **Critic timing (Stage 2).** The Critic rewards a dish served on the earliest possible turn (+10) and punishes any delay (-10). In the Stage 2 victory, on turn 13 the Waiter serves the Critic on time (64 → 74) and lets the newly arrived C5 wait one turn. In the defeat run the Critic, losing 12 per turn, walks out.
- **Happy Hour versus serving (Stage 3).** On turn 13 the Critic's 3-unit combo is ready just as C5 arrives with a 3-unit combo. Serving the Critic first would leave C5 unable to pay before turn 18. The player switches on Happy Hour, lets the Host seat C5, takes C5's order at 20% off and serves the Critic one turn late. The +15 recovery on turns 13 and 14 outweighs the Critic's -10 (recorded satisfaction 72), and all five customers pay: victory with $92.58 and an average of 86.0. Wasting Happy Hour on turn 1 and neglecting the kitchen leads to defeat with no payments.

- **Revenue against satisfaction.** Happy Hour cuts prices by 20%, so using it on expensive orders lowers revenue. Its value is keeping waiting customers above the walk-out line.

6. Evolution across the three stages

Each stage is a separate tagged, runnable commit. Each stage's code builds, passes the rule checks that existed at that stage, and reproduces its own victory and defeat. We checked this by checking out each tag into a clean folder. The README lists the new and changed files for each transition with reasons and compare links.

Transition	New source files	Changed source files (reason)
1 → 2	`VIPCustomer`, `CriticCustomer`	`Scenario`: setup registers a VIP on turn 4 and a Critic on turn 10. `Game`: the serve message shows satisfaction before → after, so the Critic's reaction is visible (no rule change). Source diff: 4 files, +147 / -3 lines.
2 → 3	`Host`, `ComboMeal`, `PriceModifier`, `HappyHour`	`Scenario`: adds three combos to the menu. `PriceCalculator`: applies a list of modifiers. `Game`: Host action, Happy Hour activation, recovery and countdown. `GameController`: Happy Hour prompt and Host step. `ConsoleView`: Happy Hour status and Host task. Source diff: 9 files, +384 / -10 lines.

No earlier rules were rewritten. `Customer`, `Order`, `Waiter`, `Chef`, `Cashier` and `Restaurant` are unchanged since Stage 1, and Stage 1 and 2 behaviour is still covered by the same rule checks at Stage 3.

7. Functional evidence

7.1 Automated rule checks (Stage 3, 40 checks)

```
PASS  Money: 90% of $27.00 is $24.30
PASS  Money: $15 x 90% x 80% is $10.80
PASS  Money: $9.00 + $15.00 = $24.00
PASS  3.3 turn 1: seated, satisfaction 92
PASS  3.3 turn 2: Pasta ordered, 1 of 2 units, satisfaction 84
PASS  3.3 turn 3: READY, satisfaction 76
PASS  3.3 turn 4: SERVED, satisfaction stays 76
PASS  3.3 turn 5: READY_TO_PAY
PASS  3.3 turn 6: PAID $18.00 with satisfaction 76, table freed
PASS  Take order at an empty table is rejected ("Table 2 is empty.")
PASS  Seating at Table 9 is rejected ("There is no Table 9. Tables are numbered 1 to 2.")
PASS  Cooking a missing order is rejected ("There is no order #5.")
PASS  Waiter can still act after rejected choices
PASS  Waiter cannot do a second task ("The Waiter has already completed a task this turn.")
PASS  Serving a dish finished this turn is rejected ("Order #1 Pasta for C1 was finished this turn. It can be
served next turn.")
PASS  The same dish can be served next turn
PASS  Regular left at end of turn 13 (100 - 13 x 8 < 0)
PASS  Departure counted, no revenue
PASS  All-wait base game ends in DEFEAT after 18 turns
PASS  VIP pays 90%: $18.00 becomes $16.20
PASS  VIP loses 5 per waiting turn
PASS  VIP order is locked at 90% of the menu price
PASS  Critic orders the most expensive item (Burger + Pasta Combo)
PASS  Critic loses 12 per waiting turn
PASS  Critic served on time: +10 (satisfaction 62)
PASS  Critic served one turn late: -10 (satisfaction 30)
PASS  Salad + Pasta Combo costs $24.30 and needs 3 units
PASS  Salad + Burger Combo costs $21.60 and needs 2 units
PASS  Most expensive item is the Burger + Pasta Combo ($29.70)
PASS  Before Happy Hour a VIP Burger costs $13.50
PASS  VIP Burger during Happy Hour: $15 x 0.9 x 0.8 = $10.80
PASS  Happy Hour cannot be activated twice ("Happy Hour can only be used once per game.")
PASS  Waiting customer recovers 15 at the start of a Happy Hour turn (84 -> 99)
PASS  Happy Hour shows 2 turns left on its first turn
PASS  Order placed before Happy Hour keeps its locked price
PASS  Happy Hour still active on its second turn
PASS  Happy Hour ends after two turns
PASS  Happy Hour cannot be used again in the same game ("Happy Hour can only be used once per game.")
PASS  Host seats C1 and the Waiter takes the order in the same turn
ALL CHECKS PASSED
```

7.2 Reproducible runs (seed 2003)

Run	Script	Paid	Revenue	Avg satisfaction	Departures	Result
Stage 1 victory	`runs/stage1-victory.txt`	5	$57.00	84.0	0	VICTORY
Stage 1 defeat	`runs/stage1-defeat.txt`	1	$15.00	84.0	1	DEFEAT
Stage 2 victory	`runs/stage2-victory.txt`	5	$65.10	81.6	0	VICTORY
Stage 2 defeat	`runs/stage2-defeat.txt`	2	$23.10	87.0	1	DEFEAT
Stage 3 victory	`runs/stage3-victory.txt`	5	$92.58	86.0	0	VICTORY
Stage 3 defeat	`runs/stage3-defeat.txt`	0	$0.00	N/A	2	DEFEAT

Full transcripts are in `runs/*-output.txt`. Selected excerpts follow.

7.3 Invalid input is explained and the task is not used up (Stage 1, turn 3)

```
Waiter: 1) Seat a customer  2) Take an order  3) Serve a READY order  0) Wait > abc
  ! "abc" is not a whole number. Please try again.
Waiter: 1) Seat a customer  2) Take an order  3) Serve a READY order  0) Wait > 3
  Order number to serve > 9
  ! Not allowed: There is no order #9.
  ! Please choose again.
Waiter: 1) Seat a customer  2) Take an order  3) Serve a READY order  0) Wait > 3
  Order number to serve > 1
  * Waiter served order #1 Burger for C1. C1 (Regular) is now eating (satisfaction 84).
```

7.4 A dish going READY → SERVED → paid (Stage 1, customer C1)

```
Turn 2  * Waiter took order #1: C1 (Regular) ordered Burger (1 prep unit) for $15.00.
        * Chef finished order #1 Burger for C1 (unit 1 of 1). It is READY and can be served next turn.
Turn 3  * Waiter served order #1 Burger for C1. C1 (Regular) is now eating (satisfaction 84).
Turn 4  * C1 (Regular) finished eating and is READY_TO_PAY (pays next turn at the earliest).
Turn 5  * Cashier collected $15.00 from C1 (Regular). Satisfaction 84 recorded; table freed.
```

7.5 VIP discount and Critic behaviour (Stage 2)

```
* Waiter took order #2: C2 (VIP) ordered Salad (1 prep unit) for $8.10.
* Waiter took order #4: C4 (Critic) ordered Pasta (2 prep units) for $18.00.
* Waiter served order #4 Pasta for C4. C4 (Critic) is now eating (satisfaction 64 -> 74 (reaction to
service)).
```

7.6 Stage 3 features: Host, combo, Happy Hour (Stage 3, turns 13-16)

```
* Happy Hour is ON for turns 13 and 14: 20% off new orders, waiting customers recover 15 each turn.
* Happy Hour: 1 waiting customer recovered 15 satisfaction (maximum 100).
Turn 13 | Revenue $43.44 | Avg satisfaction (paid) 91.3 | Paid 3 | Happy Hour ON (2 turns left incl. this
one)
  Table 2: C4 (Critic)    sat  79  ORDERED      order #4 Burger + Pasta Combo READY
* Host seated C5 (Regular) at Table 1.
* Waiter took order #5: C5 (Regular) ordered Salad + Pasta Combo (3 prep units) for $19.44
  (menu price $24.30, Happy Hour applied).
Turn 14 | ... | Happy Hour ON (1 turn left incl. this one)
* Waiter served order #4 Burger + Pasta Combo for C4. C4 (Critic) is now eating
  (satisfaction 82 -> 72 (reaction to service)).
Turn 16
Host: 1) Seat a customer  0) Wait > 1
  ! Not allowed: Table 2 is occupied by C4 (Critic).
```

7.7 Final results (Stage 3)

```
       *** VICTORY ***
[MET]    Paid customers: 5 (target 4)
[MET]    Revenue: $92.58 (target $45.00)
[MET]    Average satisfaction: 86.0 (target 60)
Dishes served: 5 | Unhappy departures: 0 | Turns used: 18/18

       *** DEFEAT ***
[MISSED] Paid customers: 0 (target 4)
[MISSED] Revenue: $0.00 (target $45.00)
[MISSED] Average satisfaction: N/A (target 60)
Dishes served: 0 | Unhappy departures: 2 | Turns used: 18/18
```

8. Additional features

- **Reproducible scripted play.** Any text file can be redirected into the game. Lines starting with # are comments, and when a script ends every remaining choice safely defaults to "Wait". `--echo` prints each answer so transcripts read like a live session. This makes the victory and defeat demonstrations repeatable.
- **Configurable seed** (`--seed N`) to try different customer orders. The default 2003 is used for all evidence.
- **Offline rule checks** (40 checks, no test library), including a turn-by-turn replay of the brief's Section 3.3 example and the brief's price examples ($24.30 combo, $10.80 VIP Happy Hour burger).
- **Helpful display**: each turn lists the useful task for every staff member, the turns left in Happy Hour, and the menu price whenever a discount applies.

- **Pixel-art window** (`run.bat --gui`): an animated graphical version of the same game, added after `stage-3`. It is a second boundary: `PixelGame` calls the same public `Game` actions as the console and `PixelRoom` only draws the current state, so no rule was changed or duplicated. Its shift log also shows each waiting customer's satisfaction loss and whether each served order was discounted.
- **Demo jumps** in the pixel-art window: `DemoJump` starts a fresh game and replays the recorded victory or defeat run to a chosen point (turn 4 VIP, turn 10 Critic, turn 13 Happy Hour decision, or either ending), so a presenter can move forwards or back and always reach the same state.

9. Reflection

Chiang Jun Siang

Chong Yan Qing:

Through this project, I learnt how to apply the object-oriented concepts and design principles taught in class, such as encapsulation, inheritance, polymorphism, composition and SOLID. I gained a better understanding of how to assign clear responsibilities to classes and separate game rules from the user interface. I also learnt that good design involves balancing simplicity with flexibility, rather than adding abstractions unnecessarily. Extending the game across three stages helped me evaluate which design decisions supported change and which needed improvement.

Choo Chong Kai Jeremy

Christopher Tan Chin Poh

This project helped me see how the object-oriented concepts from class fit together in an actual program, instead of learning them one at a time. The clearest example for me was Stage 2: the VIP and Critic customers each handle their own behaviour, so the game can treat every customer the same way and still get different results, which is what polymorphism means in practice. I also learnt why it helps to give each class one clear job and to keep the game rules separate from the user interface, because it made later changes easier. In future projects, I would spend more time on the class design at the start, since a good structure early on saves time spent reworking code later